شرح سطر بسطر ٥ — كتالوج المحتوى (مواد ← وحدات ← دروس ← مفاهيم)

مشروع رِزمة · Flutter · كل سطر مع سبب كتابته بهذه الطريقة

الواجهة لا تكتب أسماء المواد والدروس في الكود؛ تقرؤها من ملف JSON هو `assets/content/catalog_grade5.json` (هيكل مبسّط للصف الخامس: 3 مواد و6 وحدات و12 درسًا). عناوينه مؤقتة وتنتظر مراجعة معلم كما يذكر الحقل `_note` في أول الملف.

هذه الملفات الأربعة تحوّل ذلك الـ JSON إلى كائنات (`Catalog` وفروعه)، وتقرؤه (`CatalogRepository`)، وتورّعه على الشاشات (`catalogProvider`)، وتربط مفتاح أيقونة المادة بملف SVG.

lib/features/catalog/catalog_models.dart — نماذج الكتالوج

lib/features/catalog/catalog_models.dart

التسلسل الهرمي: `CatalogConcept` ← `CatalogLesson` ← `CatalogUnit` ← `CatalogSubject` ← `Catalog`. كل صنف غير قابل للتغيير (`const` وحقول `final`).

السطر 1

```
1  import '../../core/db/enums.dart';
```

ماذا يفعل؟ استيراد تعداد `ProfileColor`.

لماذا بهذه الطريقة؟ لون المادة يُستعمل نفس تعداد ألوان الطالب لنتّخذ منه ألوان الواجهة.

الأسطر 3–4

```
3  class CatalogConcept {
4    const CatalogConcept({required this.id, required this.title});
```

ماذا يفعل؟ صنف `CatalogConcept` وبانيه (المعرّف والعنوان).

لماذا بهذه الطريقة؟ المفهوم أصغر وحدة: مثل «القيمة المنزلية». عليه تُقاس «الإتقان».

الأسطر 6–11

```
6    factory CatalogConcept.fromJson(Map<String, dynamic> json) {
7      return CatalogConcept(
8        id: json['id'] as String,
9        title: json['title'] as String,
10     );
11   }
```

ماذا يفعل؟ `fromJson`: يبني المفهوم من خريطة JSON.

لماذا بهذه الطريقة؟ `as String` تحويل صريح؛ لو كان الحقل مفقودًا أو من نوع خاطئ يفشل مبكرًا بدل بيانات صامتة الخطأ.

الأسطر 13–15

```
  final String id;
  final String title;
}
```

ماذا يفعل؟ حقلا المفهوم.

لماذا بهذه الطريقة؟ غير قابلين للتغيير.

الأسطر 17–26

```
class CatalogLesson {
  const CatalogLesson({
    required this.id,
    required this.title,
    required this.estimatedMinutes,
    required this.order,
    required this.concepts,
    required this.subjectId,
    required this.unitId,
  });
```

ماذا يفعل؟ صنف `CatalogLesson` وبانيه بسبعة حقول مطلوبة.

لماذا بهذه الطريقة؟ يحمل `subjectId` و`unitId` ليعرف الدرسُ مادتَه ووحدتَه دون البحث.

الأسطر 28–32

```
  factory CatalogLesson.fromJson(
    Map<String, dynamic> json, {
    required String subjectId,
    required String unitId,
  }) {
```

ماذا يفعل؟ `fromJson` للدرس: خريطة JSON ثم المادة والوحدة كمعاملين.

لماذا بهذه الطريقة؟ المادة والوحدة ليستا في JSON الدرس نفسه (يعرفهما الأب) فتُمرَّران من الأب.

الأسطر 33–37

```
    return CatalogLesson(
      id: json['id'] as String,
      title: json['title'] as String,
      estimatedMinutes: json['estimatedMinutes'] as int,
      order: json['order'] as int,
```

ماذا يفعل؟ نقرأ المعرّف والعنوان والمدة التقديرية والترتيب.

لماذا بهذه الطريقة؟ `estimatedMinutes` يظهر في بطاقة «ابدأ أول درس».

الأسطر 38–41

```
        concepts: [
          for (final c in json['concepts'] as List<dynamic>)
            CatalogConcept.fromJson(c as Map<String, dynamic>),
        ],
```

ماذا يفعل؟ نبني قائمة المفاهيم بحلقة `for` داخل القائمة.

لماذا بهذه الطريقة؟ صيغة مختصرة بدل `map().toList()`.

الأسطر 42–45

```
        subjectId: subjectId,
        unitId: unitId,
      );
    }
```

ماذا يفعل؟ نمرّر `subjectId` و`unitId` ونغلق.

لماذا بهذه الطريقة؟ تُخزَّنان في الدرس.

الأسطر 47–54

```
    final String id;
    final String title;
    final int estimatedMinutes;
    final int order;
    final List<CatalogConcept> concepts;
    final String subjectId;
    final String unitId;
  }
```

ماذا يفعل؟ حقول الدرس.

لماذا بهذه الطريقة؟ غير قابلة للتغيير.

الأسطر 56–62

```
  class CatalogUnit {
    const CatalogUnit({
      required this.id,
      required this.title,
      required this.order,
      required this.lessons,
    });
```

ماذا يفعل؟ صنف `CatalogUnit` وبانيه.

لماذا بهذه الطريقة؟ الوحدة تجمع دروسًا.

الأسطر 64–67

```
64    factory CatalogUnit.fromJson(
65      Map<String, dynamic> json, {
66      required String subjectId,
67    }) {
```

ماذا يفعل؟ `fromJson` للوحدة: خريطة JSON ومعرّف المادة.

لماذا بهذه الطريقة؟ الأب (المادة) يمرّر معرّفه.

الأسطر 68–76

```
68      final id = json['id'] as String;
69      final lessons = [
70        for (final l in json['lessons'] as List<dynamic>)
71          CatalogLesson.fromJson(
72            l as Map<String, dynamic>,
73            subjectId: subjectId,
74            unitId: id,
75          ),
76      ]..sort((a, b) => a.order.compareTo(b.order));
```

ماذا يفعل؟ نقرأ المعرّف ونبني الدروس ونرتّبها بحقل `order`.

لماذا بهذه الطريقة؟ الترتيب من JSON لا من ترتيب الملف، فلا يفسده إعادة ترتيب أسطره. `..sort` تُرتّب القائمة في مكانها وتعيدها.

الأسطر 77–83

```
77      return CatalogUnit(
78        id: id,
79        title: json['title'] as String,
80        order: json['order'] as int,
81        lessons: lessons,
82      );
83    }
```

ماذا يفعل؟ نبني الوحدة ونعيدها.

لماذا بهذه الطريقة؟ ترتيب الوحدة نفسها أيضًا من JSON.

الأسطر 85–89

```
85    final String id;
86    final String title;
87    final int order;
88    final List<CatalogLesson> lessons;
89  }
```

ماذا يفعل؟ حقول الوحدة.

لماذا بهذه الطريقة؟ غير قابلة للتغيير.

الأسطر 91–99

```
class CatalogSubject {
  const CatalogSubject({
    required this.id,
    required this.title,
    required this.icon,
    required this.color,
    required this.order,
    required this.units,
  });
```

ماذا يفعل؟ صنف `CatalogSubject` وبانيه.

لماذا بهذه الطريقة؟ المادة تجمع وحدات ولها أيقونة ولون.

الأسطر 101–106

```
  factory CatalogSubject.fromJson(Map<String, dynamic> json) {
    final id = json['id'] as String;
    final units = [
      for (final u in json['units'] as List<dynamic>)
        CatalogUnit.fromJson(u as Map<String, dynamic>, subjectId: id),
    ]..sort((a, b) => a.order.compareTo(b.order));
```

ماذا يفعل؟ `fromJson` للمادة: نقرأ المعرّف ونبني الوحدات ونرتّبها.

لماذا بهذه الطريقة؟ نفس نمط الوحدة.

الأسطر 107–115

```
    return CatalogSubject(
      id: id,
      title: json['title'] as String,
      icon: json['icon'] as String,
      color: ProfileColor.values.byName(json['color'] as String),
      order: json['order'] as int,
      units: units,
    );
  }
```

ماذا يفعل؟ نبني المادة؛ اللون يُحوَّل من النص إلى تعداد بـ `byName` .

لماذا بهذه الطريقة؟ `byName` يرمي خطأً إن كان اللون غير معروف فنكتشف خطأ JSON مبكرًا.

الأسطر 117–118

```
  final String id;
  final String title;
```

ماذا يفعل؟ المعرّف والعنوان.

لماذا بهذه الطريقة؟ العنوان نص عربي من JSON.

الأسطر 120–124

```
  /// مفتاح الأيقونة (مثل calc / book / chat)، والواجهة هي من يحوّله لملف SVG.
  final String icon;
  final ProfileColor color;
  final int order;
  final List<CatalogUnit> units;
```

ماذا يفعل؟ مفتاح الأيقونة واللون والترتيب والوحدات.

لماذا بهذه الطريقة؟ الكتالوج يحدّد «ما هي المادة»، والواجهة تحدّد «كيف تُرسم» (الأيقونة).

الأسطر 126–128

```
  /// كل دروس المادة بترتيب الوحدات ثم ترتيب الدرس.
  List<CatalogLesson> get lessons => [for (final u in units) ...u.lessons];
}
```

ماذا يفعل؟ `lessons`: كل دروس المادة بترتيب الوحدات ثم الدروس.

لماذا بهذه الطريقة؟ `...` تفرد قائمة كل وحدة داخل قائمة واحدة.

الأسطر 130–131

```
/// كتالوج المحتوى لصف واحد: مواد ← وحدات ← دروس ← مفاهيم.
class Catalog {
```

ماذا يفعل؟ تعليق وصنف `Catalog`.

لماذا بهذه الطريقة؟ يمثّل كتالوج صف واحد.

الأسطر 132–138

```
  Catalog({required this.grade, required List<CatalogSubject> subjects})
    : subjects = List.unmodifiable(subjects),
      _lessonsById = {
        for (final s in subjects)
          for (final l in s.lessons) l.id: l,
      },
      _subjectsById = {for (final s in subjects) s.id: s};
```

ماذا يفعل؟ الباني: يجعل قائمة المواد غير قابلة للتعديل ويبني فهرسين سريعين للدروس والمواد.

لماذا بهذه الطريقة؟ الفهارس (Map) تجعل البحث بالمعرّف O(1) بدل البحث في قوائم متداخلة في كل مرة.

الأسطر 140–146

```
  factory Catalog.fromJson(Map<String, dynamic> json) {
    final subjects = [
      for (final s in json['subjects'] as List<dynamic>)
        CatalogSubject.fromJson(s as Map<String, dynamic>),
    ]..sort((a, b) => a.order.compareTo(b.order));
    return Catalog(grade: json['grade'] as int, subjects: subjects);
  }
```

ماذا يفعل؟ `fromJson` للكتالوج: نبني المواد ونرتّبها ونُنشئ `Catalog`.

لماذا بهذه الطريقة؟ نقطة الدخول من JSON.

الأسطر 148–151

```
  final int grade;
  final List<CatalogSubject> subjects;
  final Map<String, CatalogLesson> _lessonsById;
  final Map<String, CatalogSubject> _subjectsById;
```

ماذا يفعل؟ الحقول: الصف والمواد والفهرسان الخاصان.

لماذا بهذه الطريقة؟ الفهرسان خاصان لأن المستعمل يستعمل دوال البحث.

الأسطر 153–156

```
  /// كل الدروس بالترتيب: مادة بعد مادة.
  List<CatalogLesson> get allLessons => [
    for (final s in subjects) ...s.lessons,
  ];
```

ماذا يفعل؟ `allLessons`: كل الدروس بالترتيب، مادة بعد مادة.

لماذا بهذه الطريقة؟ منها نأخذ «أول درس».

الأسطر 158–161

```
  CatalogLesson? get firstLesson {
    final lessons = allLessons;
    return lessons.isEmpty ? null : lessons.first;
  }
```

ماذا يفعل؟ `firstLesson`: أول درس أو `null` إن كان الكتالوج فارغًا.

لماذا بهذه الطريقة؟ لشاشة «أول يوم».

الأسطر 163–165

```
  CatalogLesson? lessonById(String id) => _lessonsById[id];

  CatalogSubject? subjectById(String id) => _subjectsById[id];
```

ماذا يفعل؟ `lessonById` و`subjectById`: بحث بالمعرّف من الفهرس.

لماذا بهذه الطريقة؟ تعيد `null` إن لم توجد.

الأسطر 167–174

```
  CatalogUnit? unitOfLesson(CatalogLesson lesson) {
    final subject = _subjectsById[lesson.subjectId];
    if (subject == null) return null;
    for (final u in subject.units) {
      if (u.id == lesson.unitId) return u;
    }
    return null;
  }
```

ماذا يفعل؟ `unitOfLesson`: تجد وحدة درس بالبحث داخل مادته.

لماذا بهذه الطريقة؟ لعرض «المادة · الوحدة».

الأسطر 176–182

```
  /// إن كان الأخير null الدرس التالي في نفس المادة، أو.
  CatalogLesson? nextLessonInSubject(CatalogLesson lesson) {
    final lessons = _subjectsById[lesson.subjectId]?.lessons ?? const [];
    final i = lessons.indexWhere((l) => l.id == lesson.id);
    if (i < 0 || i + 1 >= lessons.length) return null;
    return lessons[i + 1];
  }
```

ماذا يفعل؟ `nextLessonInSubject`: الدرس التالي في نفس المادة أو `null` إن كان الأخير.

لماذا بهذه الطريقة؟ تُستعمل لاختيار الدرس التالي في بطاقة المتابعة.

الأسطر 184–193

```
  /// المفهوم ودرسه من معرّف المفهوم.
  ({CatalogConcept concept, CatalogLesson lesson})? conceptById(String id) {
    for (final lesson in _lessonsById.values) {
      for (final c in lesson.concepts) {
        if (c.id == id) return (concept: c, lesson: lesson);
      }
    }
    return null;
  }
}
```

ماذا يفعل؟ `conceptById`: تعيد المفهوم ودرسه (سجل مسمّى) من معرّف المفهوم ثم إغلاق الصنف.

لماذا بهذه الطريقة؟ قسم المراجعة يعرف معرّف المفهوم فقط من جدول الإتقان فيحتاج اسمه من الكتالوج. السجل يعيد قيمتين دون صنف إضافي.

lib/features/catalog/catalog_repository.dart — قراءة الكتالوج

`lib/features/catalog/catalog_repository.dart`

الأسطر 1–5

```
import 'dart:convert';

import 'package:flutter/services.dart';

import 'catalog_models.dart';
```

ماذا يفعل؟ استيراد `dart:convert` وخدمات Flutter ونماذج الكتالوج.

لماذا بهذه الطريقة؟ `rootBundle` لقراءة ملفات الأصول، و`jsonDecode` لتحليل JSON.

الأسطر 7–10

```
/// يقرأ كتالوج المحتوى من ملف JSON ضمن الأصول.
/// يُقرأ مرة واحدة ويُحفظ؛ حجم الملف الحالي صغير. إن كبر لاحقًا يُنقل
/// التحليل إلى `compute` (isolate) حتى لا يعطّل الواجهة.
class CatalogRepository {
```

ماذا يفعل؟ تعليق وصنف `CatalogRepository` .

لماذا بهذه الطريقة؟ يُقرأ مرة واحدة ويُحفظ؛ حجم الملف صغير. إن كبر يُنقل التحليل إلى (isolate) `compute` حتى لا يعطّل الواجهة.

الأسطر 11–14

```
  CatalogRepository({
    Future<String> Function(String path)? loadString,
    this.assetPath = 'assets/content/catalog_grade5.json',
  }) : _loadString = loadString ?? rootBundle.loadString;
```

ماذا يفعل؟ الباني: دالة قراءة نص اختيارية ومسار الملف الافتراضي.

لماذا بهذه الطريقة؟ حقن دالة القراءة يتيح للاختبارات قراءة الملف من القرص دون `rootBundle` .

الأسطر 16–18

```
  final String assetPath;
  final Future<String> Function(String path) _loadString;
  Catalog? _cached;
```

ماذا يفعل؟ الحقول: المسار والدالة والكتالوج المحفوظ مؤقتًا.

لماذا بهذه الطريقة؟ `_cached` يحفظ النتيجة بعد أول قراءة.

الأسطر 20–22

```
  Future<Catalog> load() async {
    final cached = _cached;
    if (cached != null) return cached;
```

ماذا يفعل؟ `load` : إن كان محفوظًا نعيده فورًا.

لماذا بهذه الطريقة؟ لا نقرأ الملف مرتين.

الأسطر 23–25

```
    final raw = await _loadString(assetPath);
    final catalog = Catalog.fromJson(jsonDecode(raw) as Map<String, dynamic>);
    return _cached = catalog;
```

ماذا يفعل؟ نقرأ النص ونحلّله ونبني `Catalog` ونحفظه ونعيده.

لماذا بهذه الطريقة؟ `_cached = catalog` تعيّن وتعيد القيمة بسطر واحد.

الأسطر 26–27

```
  }
}
```

ماذا يفعل؟ إغلاق الدالة والصنف.

لماذا بهذه الطريقة؟ نهاية الملف.

lib/features/catalog/catalog_providers.dart — مزوّدا الكتالوج

lib/features/catalog/catalog_providers.dart

الأسطر 1–4

```
import 'package:flutter_riverpod/flutter_riverpod.dart';

import 'catalog_models.dart';
import 'catalog_repository.dart';
```

ماذا يفعل؟ استيراد Riverpod والنماذج والمستودع.

لماذا بهذه الطريقة؟ ما يلزم للمزوّدين.

الأسطر 6–8

```
final catalogRepositoryProvider = Provider<CatalogRepository>((ref) {
  return CatalogRepository();
});
```

ماذا يفعل؟ `catalogRepositoryProvider` ينشئ المستودع.

لماذا بهذه الطريقة؟ يُستبدل في الاختبارات بنسخة تقرأ نصًّا جاهزًا.

الأسطر 10–12

```
final catalogProvider = FutureProvider<Catalog>((ref) {
  return ref.watch(catalogRepositoryProvider).load();
});
```

ماذا يفعل؟ `catalogProvider` من نوع `FutureProvider<Catalog>`.

لماذا بهذه الطريقة؟ القراءة غير متزامنة؛ الشاشات تتعامل مع حالات التحميل/الخطأ/القيمة (`AsyncValue`).

lib/features/catalog/subject_visuals.dart — أيقونات المواد

lib/features/catalog/subject_visuals.dart

الأسطر 1–4

```dart
import '../../core/constants/app_assets.dart';

/// يحوّل مفتاح أيقونة المادة في الكتالوج (calc / book / chat / leaf) إلى ملف SVG.
/// الكتالوج يحدّد «ما هي المادة»، والواجهة تحدّد «كيف تُرسم».
```

ماذا يفعل؟ استيراد الأصول وتعليق الغرض.

لماذا بهذه الطريقة؟ نفصل «ما هي المادة» (في JSON) عن «كيف تُرسم» (الأصول).

الأسطر 5–6

```dart
String subjectIconAsset(String key) {
  return switch (key) {
```

ماذا يفعل؟ دالة `subjectIconAsset(key)` و`switch` تعبيرية على المفتاح.

لماذا بهذه الطريقة؟ `switch` الحديثة أوضح من سلسلة `if`.

الأسطر 7–10

```dart
    'calc' => AppAssets.calcSubject,
    'book' => AppAssets.bookSubject,
    'chat' => AppAssets.chat,
    'leaf' => AppAssets.leaf,
```

ماذا يفعل؟ المفاتيح الأربعة المعروفة: حاسبة وكتاب ومحادثة وورقة.

لماذا بهذه الطريقة؟ كل مفتاح ← ملف SVG.

الأسطر 11–13

```dart
    _ => AppAssets.bookSubject,
  };
}
```

ماذا يفعل؟ الحالة الافتراضية: أيقونة الكتاب. ثم إغلاق الدالة.

لماذا بهذه الطريقة؟ مادة جديدة بمفتاح غير معروف لا تكسر الواجهة.